

PHASE 2 FINAL PROJECT REPORT

Mobile IMU Telemetry Controlled TurtleBot3

Domain: Robotics Systems, Simulation & Control
Task: Mobile IMU Telemetry Controlled TurtleBot3
Team: Simulation, Navigation & Data Collection



Contents

1	Project Objective	2
2	System Architecture	3
3	Development Timeline & Methodology	4
3.1	Week 1: Infrastructure & Data Acquisition	4
3.2	Week 2: ROS 2 Integration & Kinematics	4
3.3	Week 3: Polish & Bonus Implementations	4
4	Key Challenges & Solutions	5
5	Conclusion & Deliverables Achieved	6

1 Project Objective

The objective of this project was to establish a hardware-in-the-loop simulation by controlling a TurtleBot3 in Gazebo using real-time Inertial Measurement Unit (IMU) data. Instead of traditional keyboard teleoperation, the system utilizes a mobile phone's internal accelerometer sensors, streamed wirelessly over a local UDP network, to dictate the robot's linear and angular velocities.

2 System Architecture

The system bridges external mobile hardware, a local ROS 2 middleware environment, a 3D physics simulator, and a web-based visualization dashboard.

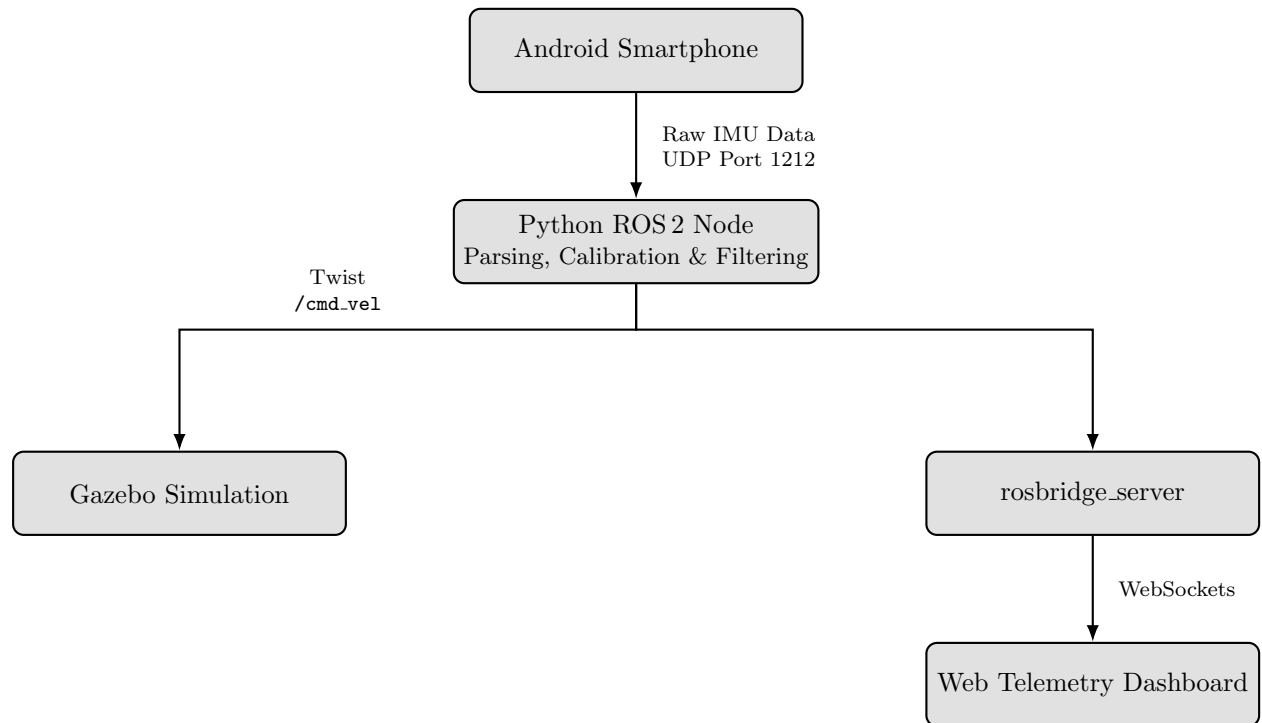


Figure 1: High-level system architecture showing data flow from the mobile sensor to the simulation and dashboard layers.

3 Development Timeline & Methodology

3.1 Week 1: Infrastructure & Data Acquisition

- **Environment Setup:** Provisioned Ubuntu 22.04 LTS and installed ROS 2 Humble Hawksbill alongside Gazebo11 and the required TurtleBot3 packages.
- **Network Bridging:** Configured the local Wi-Fi network and firewall (`ufw`) to allow incoming UDP packets on port 1212.
- **Data Ingestion:** Wrote a foundational Python script using the `socket` library to bind to the port, receive the UDP datagrams from the VirtualIMU Android application, and parse the delimited UTF-8 string into discrete accelerometer values.

3.2 Week 2: ROS 2 Integration & Kinematics

- **Node Creation:** Migrated the standalone Python script into a fully functional ROS 2 `rclpy` node within a custom `mobile_imu_control` package.
- **Kinematic Mapping:** Implemented mathematical transformations using `math.atan2` to convert raw accelerometer data (m/s^2) into Pitch and Roll angles.
- **Velocity Publishing:** Mapped the calculated angles to a `geometry_msgs/Twist` message, publishing to the `/cmd_vel` topic to achieve real-time differential drive control.

3.3 Week 3: Polish & Bonus Implementations

- **Signal Processing:** Implemented a Simple Moving Average (SMA) filter to smooth out noisy raw sensor data and a “Deadzone” to ignore micro-movements, resulting in fluid, jitter-free robot control.
- **Calibration System:** Developed a 2-second startup routine that samples the phone’s resting angle and establishes it as the absolute zero point to prevent drifting.
- **Telemetry Dashboard:** Built a full-stack HTML/JS web dashboard utilizing `rosbridge_suite` and WebSockets to visualize the robot’s real-time velocity metrics.

4 Key Challenges & Solutions

Challenge		Solution
Network Firewall Restrictions		Initial UDP packets were dropped silently. Utilized <code>netcat</code> to debug the network layer and explicitly configured <code>ufw</code> to open the required ports.
Coordinate Mismatch	System	Standard aeronautical math caused the robot to move incorrectly because Android hardware utilizes a vertical Y-axis. Customized the kinematic algorithms to invert and swap the axes, ensuring natural human-to-robot control mapping.
Web-to-ROS Communication	Com-	Standard web browsers cannot read ROS topics natively. Integrated the <code>rosbridge_server</code> package to translate ROS 2 data into WebSockets, allowing a standard HTML/JS frontend to subscribe to the <code>/cmd_vel</code> topic.

Table 1: Summary of major technical challenges encountered and their corresponding solutions.

5 Conclusion & Deliverables Achieved

All minimum requirements and 100% of the suggested Bonus Ideas were successfully completed. The final system demonstrates robust network handling, accurate kinematic transformations, real-time signal filtering, and full-stack integration between physical hardware, robotic middleware, and web technologies.

Final Deliverables

1. Source Code & ROS 2 Package (GitHub Repository):
`https://github.com/ras-al/Mobile-IMU-Telemetry-Controller-for-TurtleBot3`
2. Project Documentation (README.md)
3. System Architecture Diagram
4. Split-screen Demonstration Video